

# Design and Implementation of Algorithms for Materialized View Selection and Maintenance in Data Warehousing Environment

Ravindra N. Jogekar<sup>1</sup>, Ashish Mohod<sup>2</sup>

<sup>1</sup> Department of Computer Science & Engineering, Priyadarshini J L College of Engineering, Nagpur, India

<sup>2</sup> MTech. 4<sup>th</sup> SEM, Computer Science & Engineering Department of Computer Science & Engineering,  
Smt. Bhagwati Chaturvedi College of Engg., Nagpur, India

**Abstract-** Data warehousing, data mining and online analytical processing represents some of the latest trends in computing environments and information technology application to large scale processing and analysis of data. Data warehousing technology is becoming essential for the effective business strategy formulation and implementation. For the success of any data warehouse, accurate and timely consolidated information along with quick and effective query response times is the basic fundamental requirement. The materialization of all views is practically impossible because of the materialized view storage space and maintenance cost constraint thus proper materialized views selection is one of the intelligent decisions in designing a data warehouse to get optimal efficiency. In this paper, we present a framework for selecting best materialized view so as to achieve the effective combination of good query response time, low query processing cost and low view maintenance cost in a specified storage space constraint. The framework implementation parameter includes query frequency cost, query storage cost and query processing cost. The framework select the best cost effective materialize views to optimize the query processing time thereby resulting efficient data warehousing system.

**Keywords-** Data Warehouse Materialization, View-Maintenance, Access Frequency, Threshold, Query processing cost.

## I. INTRODUCTION

A data warehouse is a repository of subjectively selected operational data which may successfully answer any complex, statistical or analytical queries. Data warehousing enables easy organization and maintenance of large data in addition to fast retrieval and analysis in the desired manner and depth required from time to time. As the data size increases continuously, the speed requirements for processing the data so as to comprehend the meaning of this data are also required to be increase significantly. The pre-computed intermediate results obtained in the query processing are stored in the data warehouse called as materialize views, to provide effective solution for the queries posted to the data warehouse, which in turn can prevent the users from accessing the base data tables [12].

Materialized views required physical storage space acts just like a cache, which is copy of the data that can be retrieved quickly. At the same time, the use of materialized views requires additional storage space and overhead of view maintenance when refreshing the data warehouse [13]. Data warehouse is capable of answering queries and performing analysis in an efficient and quick manner, in the view of the fact that integrated information is directly available at the warehouse with differences already resolved [14]. Though the data warehouse research community provides effective solutions for the problem of representing data in a form suitable for analytical queries, it has not completely addressed other performance issues such as, query response time for a given aggregated query, view maintenance time, etc.[2]

The process of reflecting changes to a materialized view in response to the changes (inserts or update or delete) in the base relation is called as 'View Maintenance' that incurs a 'View Maintenance Cost'. Because of view maintenance cost, it is impossible to make all views materialized under the limited space constraints. This need to select an appropriate set of views to materialize for answering queries, this was denoted Materialized View Selection (MVS) and maintenance of the selected view denoted Maintenance of Materialized View (MMV). [3]

Materialized views are very important for improving performance in many business applications that's why recently database research community paying attention to the materialized view selection and maintenance. The primary intent of this research is to develop a framework for selecting views to materialize so as to achieve finer query response in low time by reducing the total cost associated with the materialized views. The proposed framework exploits all the cost metrics coupled with materialized views such as query execution frequency, query access cost, base-relation update frequency, view maintenance cost and the system's storage space constraints. The framework sustains existing materialized views periodically by removing views with low access frequency and high storage space.

The queries with high access frequencies are selected for the view selection problem.

This paper is organized as follows. We describe a related work of materialized view selection and materialized view maintenance in section 2, Materialized Views Selection framework implementation details is explaining in section 3. In section 4, we shown experimental result, and its discussion, in section 5, we concluded the paper and section 6 is used to provide the references.

## II. RELATED WORK

The problem of finding appropriate views to materialize to answer frequent queries has been studied under the name of Materialized View Selection (MVS).

Dr. T.Nalini et al. [1] proposes an IM-LXI index for incremental maintenance of materialized view selection of materialized views so that query evaluation costs can be optimized as well as view maintenance and view storage was addressed in this piece of work.

Ashadevi, B and Balasubramanian[2] proposed framework for selecting views to materialize which takes in to account all the cost metrics associated with the materialized views selection, including query processing frequencies, base relation, update frequencies, query access costs, view maintenance costs and the system's storage space constraints and then selects the most cost effective views to materialize and thus optimizes the maintenance storage, and query processing cost. This piece of work also addressed the preservation of existing materialized view.

Himanshu Gupta and Inderpal SinghMumick [3] developed a greedy algorithm to minimize the maintenance cost and storage constraint in the selection of materialized views for data warehouse. In this paper view selection under disk space & maintenance cost constraints are addressed.

Yang, J et al.[4] proposed a heuristics algorithm based on individual optimum query plans. Framework is based on specification of multiple views processing plan (MVPP), which is used to present the problem formally.

Harinarayan et al. [5] proposed a greedy algorithm for the materialized views selection so that query evaluation costs can be optimized in the special case of "data cubes". This paper provides good trade-offs between the space used and the average time to answer query. Here, the costs for view maintenance and storage were not addressed in this piece of work.

Amit Shukla et al. [6] proposed a very simple and fast heuristic algorithm, PBS, to select aggregates for pre computation. PBS algorithm runs faster than BPUS, and is fast enough to make the exploration of the time-space trade-off feasible during system configuration.

P. P. Karde et.al. [8] This paper gives an overview of various techniques that are implemented in past recent for selection of materialized view. The issues related to maintaining the materialized view are also discussed in this paper. Here some future aspects are also stated that might be useful for recent researchers.

Y.D.Choudhari et al.[11] proposed a novel CBFSMV algorithm is proposed for selection of materialized view using query clustering strategy that reduces the execution time as compared to response time for actual database.

Our main objective is to materialize the effective candidate views by taking into consideration of query frequency, query processing cost and space requirement along with view maintenance cost.

## III. MATERIALIZED VIEWS SELECTION FRAMEWORK

This section elaborates the created framework approach for the selection of materialized view. Materialized views are beneficial for the users to quickly get the search results for frequent queries. The ultimate aim behind the proposed materialized view selection framework is to materialize the user views by taking into consideration of query frequency, query processing cost and storage requirement of query. The developed framework is applied on data warehouse model, DW and a user's selected query file (UQF) that contains the list of queries used by the number of users. As it is practically impossible to create materialized view of all user queries due to the storage space constraints the queries that are frequently used by the users should be selected but, at the same time, the query processing cost and storage cost should be less. Accordingly, we have used the data warehouse, DW that contains four tables. The schema of the data warehouse used in the framework is represented with four various tables such as tblstudent (T1), tblmarks (T2), tblattendance (T3) and tblextraactivity (T4) The student table, which consists of following field records such as srno, studId, name, college branch and course where, studId is the primary key. The mark table contains one tuple for each subject marks, and its key is studId.

The attendance table contains details about the student attendance of each subject and its field records are srno, studId (foreign key), subject, class\_date and attendance. The last extraactivity table contains each student extraactivity record using following fields srno, studId (foreign key), activity\_name, student\_post and extra\_point.

The first phase of materialized view selection is generation of huge random set of records for the above given database tables using random data insertion record generator. After that all possible set of complex queries are generated on above created records. The most prominent queries are selected from the given created query set using following algorithms.

*Assumptions:*

$Q_{SET} \rightarrow$  Given set of queries  
 $Q_{AF} \rightarrow$  Query access frequency  
 $Q_{SP} \rightarrow$  Query storage space  
 $Q_{PT} \rightarrow$  Query processing time  
 $Q_{IL} \rightarrow$  Query information list  
 $MV_{SP} \rightarrow$  Materialized View storage space  
 $Q_{ID} \rightarrow$  Query information DTO  
 $T_S \rightarrow$  MV storage threshold value

**3.1. Algorithm 1**

begin:

Calculate  $MV_{SP}$

if ( $MV_{SP} < T_S$ ) then

Repeat for I  $\leftarrow 1$  to  $Q_{SET}$

$Q_{ID} \leftarrow$  find  $Q_{AF}$ ;

$Q_{ID} \leftarrow$  find  $Q_{SP}$ ;

$Q_{ID} \leftarrow$  find  $Q_{PT}$ ;

$Q_{IL} \leftarrow Q_{ID}$ ;

end repeat

end if

else

Discard the materialize view creation process.

end else

end

The above first algorithm is used to find candidate queries frequency, processing time and storage space information. In the first step how much storage space is occupied by the materialized view will be calculated and if calculated materialized view storage space value is greater than the allotted materialized view storage value then in that case materialize view creation process discarded due to storage space constraints, otherwise for each query in query set ( $Q_{SET}$ ) find the frequency, processing time and storage space that can be stored in query information list ( $Q_{IL}$ ) in the form of query information DTO ( $Q_{ID}$ ).

The second algorithm is used to find the best queries that need to materialize to optimize the complex query processing time. Here the first algorithm output i.e.  $Q_{IL}$  is used as an input to calculate the query frequency, query processing and query storage cost. Using weighted combination of query frequency, query processing and query storage cost, materialized view selection cost will be calculated for each query. After calculating each query selection cost the next step is to find the appropriate materialized view selection threshold value using summation of all the selection cost divided by number of selected queries. If the selection cost is greater than the materialized view selection threshold value then calculate materialized view storage space ( $MV_{SP}$ ) and add the selected query storage space to get the total materialized view storage space and if total materialized view storage space is less than the materialized view storage threshold value then build the materialized view for the selected query otherwise neglect the query.

*Assumptions:*

$T \rightarrow$  MV selection threshold value

$Q_{MFreq} \rightarrow$  Maximum query frequency

$Q_S \rightarrow$  Selected query storage

$Q_{MPT} \rightarrow$  Maximum query processing time

$Q_{MS} \rightarrow$  Maximum Query storage

$Q_{PC} \rightarrow$  Query processing cost

$Q_{SC} \rightarrow$  Query storage cost

$Q_{FC} \rightarrow$  Query frequency cost

$Q_{CT} \rightarrow$  Query cost table

$S_Q \rightarrow$  Query selection cost

$M_T \rightarrow$  Minimum threshold

$N \rightarrow$  Number of rows in query cost table

$N_{QIC} \rightarrow$  Number of rows in query information list

$w_1, w_2$  &  $w_3 \rightarrow$  Weighted constant values in between 0 to 1

**3.2 Algorithm 2**

begin:

Repeat for I  $\leftarrow 0$  to  $N_{QIC} - 1$

$Q_{ID} \leftarrow Q_{IL}[i]$

$Q_{AF} \leftarrow Q_{ID}$

$Q_{PT} \leftarrow Q_{ID}$

$Q_{SP} \leftarrow Q_{ID}$

$Q_{FC} \leftarrow Q_{AF} / Q_{MFreq}$ ;

$Q_{PC} \leftarrow Q_{PT} / Q_{MPT}$ ;

$Q_{SC} \leftarrow Q_{SP} / Q_{MS}$ ;

$Q_{CT} \leftarrow Q_{FC}$ ;

$Q_{CT} \leftarrow Q_{PC}$ ;

$Q_{CT} \leftarrow Q_{SC}$ ;

end repeat

[Find selection cost]

```

Repeat for I ← 1 to QCT
  SQ = w1 * QFC + w2 * QPC + w3 (1 - QSC) ;
  QCT ← SQ;
end repeat
[Select MV Selection Threshold]
MT =  $\sum_{i=K}^K S_Q / N$ 
[Select materialized view having good query response, low
processing and storage cost]
Repeat for i ← 0 to N-1
  SQ ← QCT [i]
  if (SQ ≥ MT) then
    Calculate MVSP
    MVSP ← MVSP + QS
    if (MVSP < TS) then
      Build the materialized view for the
      selected query
    else
      Discard the query
    else
      Discard the query
  end repeat

```

Thus, the above algorithms can be used for selection of effective materialize views that can be achieved the desired multi-objective i.e. it provides the best combination of better query response, low query processing cost and low storage space cost.

### 3.3. Preservation of Existing Materialized Views

This section details the preservation procedure of the already created materialized views. Before selecting new views for materialization, the existing materialized views are needed to be sustained based on their access frequency and storage space. The steps for the above process are given in Algorithm1.

*Assumptions:*

$A_{LMV}$  → Array List of Materialized views  
 $T_p$  → Preservation threshold value  
 $P_Q$  → Materialized views preservation cost  
 $Q_{SC}$  → Query storage cost  
 $Q_{FC}$  → Query frequency cost  
 $\alpha, \beta$  → Weighted constant values in between 0 to 1

*Algorithm 1:*

```

for each Materialized View in ALMV
  PQ =  $\alpha * Q_{FC} + \beta (1 - Q_{SC})$ ;
  if (PQ < Tp) then
    Remove current Materialized view;
  end if
end for

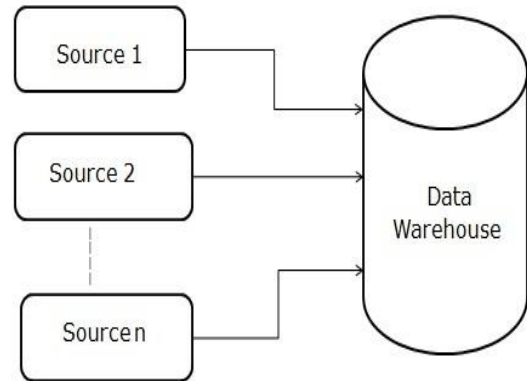
```

The above existing materialized view preservation algorithm retain the materialized views having high access frequency and low storage cost and remove the worst materialized view having low access frequency and high storage space for the materialization of new views.

## IV. MATERIALIZED VIEWS MAINTAINANCE FRAMEWORK

The proposed second approach is to handle the maintenance problem without recomputing the materialized views. When data source changes, the materialized views in data warehouse are also updated in order to maintain the consistency, this causes the need for handling the problem of materialized view maintenance. [8]

In Fig. 1 shown that there is communication between Source and the data warehouse, when update occurs at source, it sends the notification to warehouse later on warehouse sends the query to source for the corresponding update as source receives the query it sends the answer to warehouse to that corresponding query.[9]



Once we generate the materialized view for the specific data records, the maintenance of materialized view is very crucial in data warehouse. Whenever source data is changing, the warehouse does not reflect the correct state of this source and has to be maintained for data consistency. Fortunately only parts of the warehouse are affected by a modification to a single source. So the obvious solution would be to reload these parts only.

*Assumptions:*

$BT_{UC}$  → Base table update count  
 $T_M$  → MV maintenance threshold value  
 $B_{VT}$  → Binary version table  
 $Q_{AT}$  → Materialized query attribute table  
 $MV_{RF}$  → Materialize views refresh flag

#### 4.1. Algorithm1

begin:

[Start the daemon thread to initiate the materialized view maintenance process]

Repeat while (true)

$MV_{RF} \leftarrow \text{false};$

if( $BT_{UC} > T_M$ )

Repeat for  $I \leftarrow 0$  to  $Q_{AT}$

Repeat for  $J \leftarrow 0$  to  $B_{VT}$

if( $B_{VT}[J] = Q_{AT}[I]$ )

$MV_{RF} \leftarrow \text{true};$

break;

end if

end repeat

if( $MV_{RF} = \text{true}$ )

Refresh Materialize View ;

break;

end if

end repeat

end repeat while

end

This section details the maintenance of existing materialized views. After selecting materialized view maintenance step is required to keep materialized view consistent. For example, if the data warehouse gets modified (insertion and deletion of data source) after selecting materialized view, the corresponding modified data source should be reflected in the existing view. In order to deal with the updating and deletion of data source, the output of the query should be given by considering the updated data records without re-computing the whole process. Accordingly, we have designed a framework for view maintenance process that initiate the daemon thread to maintain the view after certain number of updates.

#### V. EXPERIMENTAL RESULTS AND DISCUSSION

The section shows the running experiment results that are carried out using simulated student database schema by applying algorithm 1 and 2. The various typical user queries along with its query frequency, storage space & processing time are shown in Table1. Whereas query frequency cost, processing cost, storage cost, selection cost and minimum materialized view selection threshold is calculated using algorithm2 and shown in Table2.

Table1

Materialized View Query Selection Parameter Information

| Query | $Q_{AF}$ | $Q_{SP}(\text{bytes})$ | $Q_{PT}(\text{ms})$ |
|-------|----------|------------------------|---------------------|
| Q1    | 5        | 16384                  | 203                 |
| Q2    | 4        | 344064                 | 0                   |
| Q3    | 3        | 65536                  | 0                   |
| Q4    | 2        | 16384                  | 31                  |
| Q5    | 1        | 16384                  | 15                  |

Table 2

Materialized View Query Selection Cost Information

| Query                     | $Q_{FC}$ | $Q_{SC}$ | $Q_{PC}$ | $S_Q$         |
|---------------------------|----------|----------|----------|---------------|
| Q1                        | 1        | 0.048    | 1        | 0.976         |
| Q2                        | 0.8      | 1        | 0        | 0.9           |
| Q3                        | 0.6      | 0.19     | 0        | 1.205         |
| Q4                        | 0.4      | 0.048    | 0.153    | 1.1           |
| Q5                        | 0.2      | 0.048    | 0.074    | 1.039         |
| Minimum Threshold Value = |          |          |          | <b>1.0440</b> |

The queries having selection cost is greater than the minimum materialized view selection threshold value need to be materialized for quick query processing as shown in Table3.

Table3

Queries that satisfy selection criteria

| Query | Selection cost ( $S_Q$ ) |
|-------|--------------------------|
| Q3    | 1.205                    |
| Q4    | 1.1                      |

Above table shows only those queries which satisfy the multiple constraints so here we are selecting only two queries having selection cost is greater than the minimum materialized view selection threshold value from the set of queries shown in Table1.

#### 5.1 Comparative Analysis

This section presents the comparative analysis of the proposed approach with the direct view access for analyzing the difference of query execution time.



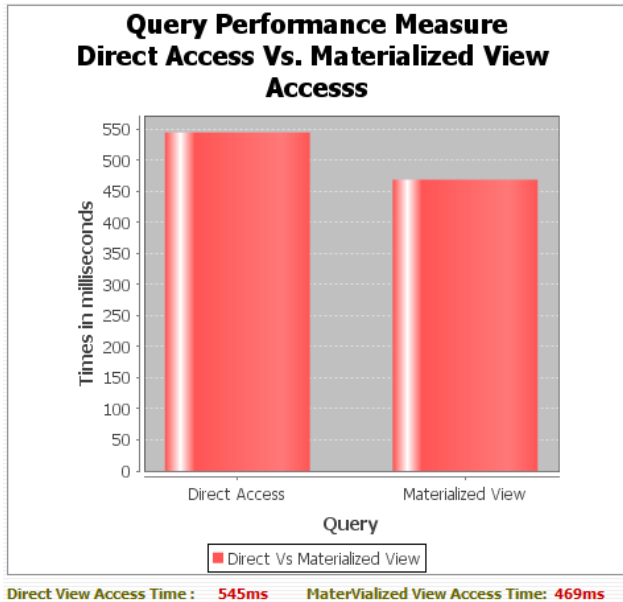
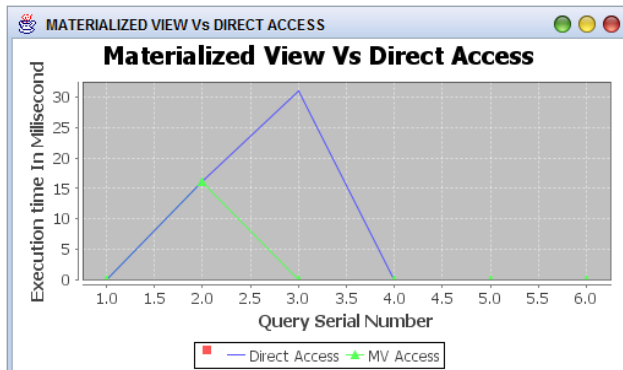


Figure 3 Shows comparison of execution time of the query using materialized view selection framework and execution time of the query if it is posted for original database (without framework).



Above snapshot represents the calculation results, from which following observations can be stated: The all-direct base table access method requires the highest query processing cost with no view maintenance and storage costs are incurred. The all-materialized-views method can provide the best query performance with some view maintenance and storage costs are incurred.

#### 5.1 Finding relevant queries to view adaptation

### VI. CONCLUSION

This paper gives the idea regarding best view selection for materialization and the effective incremental batch approach for materialized view maintenance.

The first approach is for materialized view selection, the essential constraints for materialize view selection are: query frequency, query processing time and storage space. The presented proposed methodology determines which queries are more beneficial using combination of query frequency, processing and storage cost for the creation of materialized view so as to achieve the quick query processing time.

For experimentation, the proposed framework is executed on the simulated student data warehouse model using bunch of queries, to find the efficiency of the proposed approach in selection of materialize view.

The second approach is for materialized view maintenance(MVM) where after certain number of updates on the base table view maintenance process start in which selected materialized view queries attributes (query attribute table records) are match with binary version table records which contains information of insert, update and delete queries fired on base table. If the query attribute table record are matched with binary version table records then materialized view maintenance is required otherwise there is no need to refresh the materialized view. This MVM approach avoids unnecessary refreshment of materialized view.

The third approach is for preserving best materialized views on the basis of high query frequency and low storage space and removes the materialized views with low access frequency and high storage space for the materialization of new views. For future research in this area could focus on validating this model against some real-world data warehouse.

### Acknowledgments

My sincere thanks to my honorable guide Prof. Manoj S. Chaudhari and others who have contributed towards the preparation of the paper.

### REFERENCES

- [1] Dr.T.Nalini, Dr.A.Kumaravel, Dr.K.Rangarajan,"A Novel Algorithm with IM-LSI Index For Incremental Maintenance of Materialized View" JCS&T Vol. 12 No. 1 April 2012
- [2] B.Ashadevi, R.Balasubramanian," Cost Effective Approach for Materialized Views Selection in Data Warehousing Environment", IJCSNS International Journal of Computer Science and Network Security, VOL.8 No.10, October 2008
- [3] Gupta, H. & Mumick, I., Selection of Views to Materialize in a Data Warehouse. IEEE Transactions on Knowledge and Data Engineering, 17(1), 24-43, 2005.
- [4] Yang, J., Karlapalem, K., and Li, Q. (1997). A framework for designing materialized views in a data warehousing environment. Proceedings of the Seventieth IEEE International Conference on Distributed Computing systems, USA, pp:458.

## International Journal of Emerging Technology and Advanced Engineering

Website: [www.ijetae.com](http://www.ijetae.com) (ISSN 2250-2459, ISO 9001:2008 Certified Journal, Volume 3, Issue 9, September 2013)

- [5] V. Harinarayan, A. Rajaraman, and J. Ullman. "Implementing data cubes efficiently". Proceedings of ACM SIGMOD 1996 International Conference on Management of Data, Montreal, Canada, pages 205--216, 1996.
- [6] A. Shukla, P. Deshpande, and J. F. Naughton, "Materialized view selection for the multidimensional datasets," in Proc. 24th Int. Conf. Very Large Data Bases, 1998, pp. 488--499.
- [7] Wang, X., Gruenwald, L., and Zhu, G. (2004). A performance analysis of view maintenance techniques for data warehouses. Data warehouse knowledge, pp:1-41.
- [8] Mr. P. P. Karde, Dr. V. M. Thakare. "Selection & Maintenance of Materialized View and Its Application for Fast Query Processing: A Survey". Proceedings of International Journal of Computer Science & Engineering Survey (IJCSES) Vol.1, No.2, November 2010
- [9] Abdulaziz S. Almazayad, Mohammad Khubeb Siddiqui. "Incremental View Maintenance: An Algorithmic Approach". Proceedings of International Journal of Electrical & Computer Sciences IJECS-IJENS Vol: 10 No: 03
- [10] Elena Baralis, Tania Cerquitelli, and Silvia Chiusano, "I-Mine: Index Support for Item Set Mining" IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 4, april 2009
- [11] Y.D. Choudhari and Dr. S. K. Shrivastava, "Cluster Based Approach for Selection of Materialized Views", International Journal of Advanced Research in Computer Science and Software Engineering, Volume 2, Issue 7, July 2012
- [12] C. Zhang, X. Yao, and J. Yang. An evolutionary Approach to Materialized View Selection in a Data Warehouse Environment. IEEE Transactions on Systems, Man and Cybernetics, vol. 31, no.3, pp. 282--293, 2001.
- [13] K. Aouiche, P. Jouve, and J. Darmont. Clustering-based materialized view selection in data warehouses. In ADBIS'06, volume 4152 of LNCS, pages 81--95, 2006
- [14] Y. Zhuge, H. Garcia-Molina, J. Hammer, and J. Widom, "View Maintenance in a Warehousing Environment." In Proceedings of the ACM SIGMOD Conference, San Jose, California, May 1995.

### Authors



**Ashish Mohod** received the B.E. degree from U.C.O.E, Umred, and State-Maharashtra, India. He is pursuing MTech. in Computer Science and Engineering from Bhagwati Chaturvedi College of Engineering, Nagpur, Maharashtra, India. He was a Software developer between 2005 to 2009. He has presented more than 5 papers, in national conferences and publishes many international journal papers. He has organized workshops, state/national project events, seminars in various organizations. His interest of research is in Materialized View Selection and Maintenance process.



**Ravindra N. Jogekar** was born in Pandhurna, Madhya Pradesh in 1979. Currently he is working as an Assistant Professor of Computer Science and Engineering department of Priyadarshini J L College of Engineering from last 2 years. His interests of research is in, Artificial Intelligence, Database and Data warehousing & mining, Computer Network, Computer Graphics, Software

Engineering, Data Structure